Foundations of Dynamic Programming: Memoization, Tabulation, Complexity Collapses, and Algorithmic Analysis of Fibonacci, Staircase, and Minimum Perfect Squares Problems

Abstract

This paper presents a formal, unified algorithmic analysis of introductory Dynamic Programming (DP) principles derived from classroom instruction. First, to eliminate mathematical ambiguity, we provide a complete reference guide explaining every asymptotic symbol, square root function, floor bracket, and minimization operator. Second, we formalize the core prerequisites for DP applicability—Optimal Substructure and Overlapping Subproblems—and contrast Top-Down Memoization with Bottom-Up Tabulation. Third, we analyze the Fibonacci sequence and Climbing Stairs problems, proving how state caching collapses naive exponential $O(2^N)$ time into linear $O(N)$ time and $O(1)$ space. Fourth, we formalize the Minimum Perfect Squares problem, establishing its recurrence $S(n) = 1 + \min_{1 \le x \le \lfloor \sqrt{n} \rfloor} S(n - x^2)$ and proving its $O(N\sqrt{N})$ tabular time and $O(N)$ space complexity. Production-grade, compilable Java implementations are provided for all discussed algorithms.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Mathematical Symbols, Formal Notation, and Accessible Reference Guide

To ensure complete clarity and accessibility for readers of all mathematical and technical backgrounds, this section provides an explicit reference guide to every mathematical symbol, set notation, asymptotic function, and calculus concept utilized throughout this research paper.

1.1 Asymptotic Bounds and Complexity Notations

- Big-O Notation ($\mathcal{O}$ or O): Upper Bound
  - Formal Definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $f(N) \le c \cdot g(N)$ for all $N \ge N_0$.
  - Plain Explanation: Represents the worst-case scenario or maximum resource limit. It guarantees that execution time or space will not exceed a multiple of $g(N)$.
  - Example in Paper: Dynamic programming reduces naive $O(2^N)$ Fibonacci calculation to linear $O(N)$ time.
- Big-Theta Notation (Θ): Tight Bound
  - Formal Definition: $f(N) = \Theta(g(N))$ if $f(N) = O(g(N))$ and $f(N) = \Omega(g(N))$.
  - Plain Explanation: Represents an exact asymptotic bound. The execution time grows at precisely the same rate as $g(N)$.
  - Example in Paper: Tabulation of the Minimum Perfect Squares problem takes $\Theta(N\sqrt{N})$ exact total operations.

1.2 Calculus Primer for Algorithm Analysis: Integrals and Fractional Exponents

Many computer science proofs use calculus notation to bound summations. If you are new to calculus, this primer explains the exact concepts used in our complexity proofs.

1) Fractional Exponents ($x^{1/2}$ and $x^{3/2}$):
   - $x^{1/2}$ is another way to write the square root $\sqrt{x}$.
   - $x^{3/2}$ is $x^1 \cdot x^{1/2} = x \cdot \sqrt{x}$. For example, $100^{3/2} = 100 \times \sqrt{100} = 100 \times 10 = 1000$.
2) Definite Integrals ($\int_a^b f(x)\,dx$) as Area Under a Curve:
   - A summation $\sum_{i=1}^{N} f(i)$ adds together individual discrete steps (like bar heights in a histogram).

- An integral $\int_a^b f(x)\,dx$ measures the total continuous area under the curve $y = f(x)$ between $x = a$ and $x = b$.
- Sums of increasing functions (like $f(x) = \sqrt{x}$) can be upper-bounded by calculating the continuous area under the curve using integration.

3) The Power Rule of Integration:

- In calculus, the area under x^k is computed using the Power Rule:

$$\int x^k\,dx = \frac{x^{k+1}}{k+1} + C$$

- Applying this rule for $k = 1/2$ (where $k + 1 = 3/2$):

$$\int x^{1/2}\,dx = \frac{x^{3/2}}{3/2} = \frac{2}{3}x^{3/2} = \frac{2}{3}x\sqrt{x}$$

This tells us that the total area under $\sqrt{x}$ up to N is exactly $\frac{2}{3}N\sqrt{N}$.

TABLE 1
Quick Reference Guide to Mathematical Notation Used in the Paper

Symbol	Name / Meaning	Plain-Language Explanation
$O(g(N))$	Big-O (Worst-case)	Upper bound on time or space (won't be worse than this).
$\Theta(g(N))$	Big-Theta (Tight bound)	Exact growth rate of the algorithm.
$\lfloor x \rfloor$	Floor function	Round down to nearest whole integer.
$\sqrt{N}$ or $N^{1/2}$	Square root	Number which when multiplied by itself equals N.
$N^{3/2}$	Fractional power	Equal to $N \times \sqrt{N}$.
$\int_a^b f(x)\,dx$	Definite Integral	Total continuous area under curve $f(x)$ from a to b.
$\min_x f(x)$	Minimization operator	Pick the smallest result among all valid options x.
$\binom{n}{k}$	Binomial coefficient	Number of combinations of selecting k items from n.
$dp[n]/dp[i][j]$	DP State Table Entry	Memoized/tabulated solution for subproblem of given state.

2 Foundations of Dynamic Programming: Principles and Paradigms

Dynamic Programming (DP) is a fundamental algorithmic design paradigm coined by Richard Bellman in the 1950s. As captured in introductory computer science pedagogy, the core philosophy of DP is encapsulated by George Santayana's famous aphorism: "Those who cannot remember the past are condemned to repeat it."

2.1 Core Prerequisites for Dynamic Programming

An optimization or counting problem can be solved using Dynamic Programming if and only if it exhibits two fundamental properties:

Definition 1 (Optimal Substructure). A problem possesses optimal substructure if an optimal solution to the global problem of size N can be constructed efficiently from optimal solutions to its smaller subproblems of size $< N$.

Definition 2 (Overlapping Subproblems). A problem exhibits overlapping subproblems if its recursive decomposition breaks the main problem into smaller subproblems that are re-evaluated multiple times across different branches of the execution tree.

2.2 Top-Down Memoization vs. Bottom-Up Tabulation

Dynamic programming manifests in two primary architectural implementations:

1) Top-Down Dynamic Programming (Memoization):
   A recursive formulation that starts at the primary target state N and works backwards down to base cases. To prevent re-computation, intermediate subproblem results are stored in a lookup table (e.g., array initialized to -1).
2) Bottom-Up Dynamic Programming (Tabulation):
   An iterative formulation that starts at the base cases (e.g., $dp[0], dp[1]$) and populates an array sequentially up to N. This eliminates call-stack overhead and enables space optimization techniques.

3 Case Study I: The Fibonacci Sequence and Exponential Complexity Collapse

The Fibonacci sequence serves as the canonical baseline problem for analyzing subproblem overlapping and complexity reduction.

TABLE 2
Comparison of Dynamic Programming Paradigms

Attribute	Top-Down (Memoization)	Bottom-Up (Tabulation)
Execution Style	Recursive with Lookup Cache	Iterative State Loop
Subproblem Evaluation	On-demand (Lazy Evaluation)	Systematic / All States
Call Stack Overhead	$O(N)$ Recursion Stack Space	None ($O(1)$ Stack Space)
State Space Reduction	Difficult	Straightforward ($O(N) \to O(1)$)

3.1 Formal Recurrence and Naive Recursion Failure

The n-th Fibonacci number $F(n)$ is defined by the linear recurrence relation:

$$F(n) = F(n-1) + F(n-2) \quad \text{for } n \geq 2$$

with base conditions $F(0) = 0$ and $F(1) = 1$.

Theorem 3 (Naive Recursive Fibonacci Complexity). Evaluating $F(n)$ via naive recursion takes $\Theta(\phi^n)$ exponential time complexity, where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (the Golden Ratio).

Proof. Let $T(n)$ be the number of additions required to compute $F(n)$. The execution tree satisfies $T(n) = T(n-1)+T(n-2)+1$. Solving this linear recurrence yields $T(n) = \Theta(\phi^n) = O(2^n)$. For $n = 1000$, $2^{1000} \approx 10^{301}$ operations, requiring far longer than the estimated age of the universe. □

3.2 Top-Down Memoized Solution

By maintaining a memoization vector $F_{\text{memo}}[0 \dots N]$ initialized to -1, each unique state $i \in [0 \dots N]$ is computed exactly once:

```
public class FibonacciMemoization {
    private static int[] memo;

    public static int fib(int n) {
        memo = new int[n + 1];
        java.util.Arrays.fill(memo, -1);
        return solve(n);
    }

    private static int solve(int n) {
        if (n <= 1) return n;
        if (memo[n] != -1) return memo[n]; // Reuse cached solution
        return memo[n] = solve(n - 1) + solve(n - 2);
    }
}
```

3.3 Bottom-Up Tabulation and $O(1)$ Space Optimization

Bottom-up tabulation populates the array iteratively in $O(N)$ time. Furthermore, because computing $dp[i]$ depends only on $dp[i-1]$ and $dp[i-2]$, the $O(N)$ space table can be compressed into two scalar state variables ($O(1)$ auxiliary space):

```
public class FibonacciOptimized {
    public static int fib(int n) {
        if (n <= 1) return n;
        int prev2 = 0; // f[0]
        int prev1 = 1; // f[1]
        int curr = 0;

        for (int i = 2; i <= n; i++) {
            curr = prev1 + prev2;
            prev2 = prev1;
            prev1 = curr;
        }
        return curr;
    }
}
```

4 Case Study II: The Climbing Stairs Problem

The Climbing Stairs problem is a fundamental dynamic programming application involving path counting and step combinations.

4.1 Problem Formalization

Given a staircase containing N steps, a person standing at ground level (step 0) can climb either 1 step or 2 steps at a time. Determine the total number of distinct ways to reach the top (step N).

4.2 Mathematical Formulation and Structural Equivalence

Let $W(n)$ represent the total number of distinct ways to reach step n.

Theorem 4 (Climbing Stairs Recurrence). To reach step n, the final step taken must be either:

1) A single jump of 1 step from step $n-1$ (with $W(n-1)$ distinct ways).
2) A double jump of 2 steps from step $n-2$ (with $W(n-2)$ distinct ways).

By the Rule of Sum in combinatorics, the total ways is:

$$W(n) = W(n-1) + W(n-2) \quad \text{for } n \geq 2$$

with base conditions $W(0) = 1$ and $W(1) = 1$.

This establishes a direct structural isomorphism to the Fibonacci sequence shifted by one index position ($W(n) = F(n+1)$).

4.3 Compilable Java Implementation

```
public class ClimbingStairs {
   // Optimal Iterative Space-Optimized Solution: O(N) Time, O(1) Space
   public static int climbStairs(int n) {
      if (n <= 1) return 1;
      int prev2 = 1; // W(0)
      int prev1 = 1; // W(1)
      int totalWays = 0;

      for (int i = 2; i <= n; i++) {
         totalWays = prev1 + prev2;
         prev2 = prev1;
         prev1 = totalWays;
      }
      return totalWays;
   }
}
```

5 Case Study III: Minimum Number of Perfect Squares

The Minimum Perfect Squares problem represents a classic optimization problem requiring state iteration across variable choices.

5.1 Problem Statement

Given a positive integer N, find the minimum count of perfect square numbers (e.g., $1^2, 2^2, 3^2, 4^2, \dots$) that sum up to N.

For example:

- For $N = 12$, $12 = 4 + 4 + 4 = 2^2 + 2^2 + 2^2 \implies$ Minimum Squares = 3.
- For $N = 13$, $13 = 4 + 9 = 2^2 + 3^2 \implies$ Minimum Squares = 2.

5.2 Mathematical Recurrence Relation

Let $S(n)$ denote the minimum number of perfect squares required to sum to n.

For any value n, we consider taking a perfect square $x^2 \leq n$ (where $1 \leq x \leq \lfloor \sqrt{n} \rfloor$). Taking x^2 leaves a remaining sum of $n - x^2$, which requires $S(n - x^2)$ squares. Minimizing over all choices of x:

$$S(n) = 1 + \min_{1 \leq x \leq \lfloor \sqrt{n} \rfloor} S(n - x^2) \quad \text{for } n \geq 1$$

with base condition $S(0) = 0$.

5.3 Pedagogical Complexity Proof: From Intuition to Calculus Integration

Theorem 5 (Perfect Squares Complexity). The bottom-up tabular implementation of the Minimum Perfect Squares problem has a time complexity of $O(N\sqrt{N})$ and space complexity of $O(N)$.

Proof. We present the complexity analysis in three complementary stages, ranging from elementary arithmetic bounds to a complete calculus integration walkthrough.

5.3.0.1 Stage 1: Elementary Arithmetic Upper Bound (No Calculus): The tabular algorithm populates an array $dp[0 \dots N]$. For state i, the inner loop executes $\lfloor \sqrt{i} \rfloor$ iterations. Summing across all states $i = 1$ to N:

$$T(N) = \sum_{i=1}^{N} \lfloor \sqrt{i} \rfloor = \sqrt{1} + \sqrt{2} + \sqrt{3} + \cdots + \sqrt{N}$$

Notice that every term in this sum is less than or equal to the final term $\sqrt{N}$. If we replace every term with $\sqrt{N}$, we obtain an upper bound:

$$T(N) \leq \underbrace{\sqrt{N} + \sqrt{N} + \cdots + \sqrt{N}}_{N \text{ terms}} = N \cdot \sqrt{N} = N\sqrt{N}$$

This immediately establishes that $T(N) = O(N\sqrt{N})$.

5.3.0.2 Stage 2: Geometric Area Approximation: Geometrically, the discrete sum $\sum_{i=1}^{N} \sqrt{i}$ represents the total area of N histogram bars of width 1 and height $\sqrt{i}$. This discrete histogram area is bounded above by the continuous area under the curve $y = \sqrt{x} = x^{1/2}$ from $x = 1$ to $x = N + 1$:

$$\sum_{i=1}^{N} \lfloor \sqrt{i} \rfloor \leq \int_{1}^{N+1} x^{1/2}\, dx$$

5.3.0.3 Stage 3: Step-by-Step Calculus Integration Walkthrough: We now compute the definite integral step-by-step using the Power Rule of Integration ($\int x^k\, dx = \frac{x^{k+1}}{k+1}$):

1) Find the Antiderivative: For $k = 1/2$, the exponent becomes $1/2 + 1 = 3/2$. Dividing by $3/2$ gives the multiplier $\frac{1}{3/2} = \frac{2}{3}$:
$$\int x^{1/2}\, dx = \frac{2}{3}x^{3/2}$$
2) Evaluate Upper and Lower Limits ($x = 1$ to $x = N + 1$):
$$\int_{1}^{N+1} x^{1/2}\, dx = \left[\frac{2}{3}x^{3/2}\right]_{1}^{N+1} = \frac{2}{3}(N+1)^{3/2} - \frac{2}{3}(1)^{3/2}$$
3) Convert Fractional Power $x^{3/2}$ back to Standard Algebra: Recall that $x^{3/2} = x \cdot \sqrt{x}$. Therefore:
$$\frac{2}{3}(N+1)^{3/2} - \frac{2}{3} = \frac{2}{3}(N+1)\sqrt{N+1} - \frac{2}{3}$$
4) Asymptotic Simplification: For large N, the dominant term is $\frac{2}{3}N\sqrt{N}$. In Big-O notation, constant factors ($\frac{2}{3}$) and lower-order terms are dropped:
$$T(N) = \Theta(N^{3/2}) = O(N\sqrt{N})$$

Auxiliary space is dominated by array $dp[0 \dots N]$, which requires $O(N)$ space. □

5.4 Complete Java Reference Implementation

Below is the complete production Java code containing both Top-Down Memoization and Bottom-Up Tabulation for the Minimum Perfect Squares problem.

```
import java.util.Arrays;

public class MinPerfectSquares {

    // Approach 1: Top-Down Memoization O(N sqrt(N)) Time, O(N) Space
    public static int numSquaresMemo(int n) {
        int[] memo = new int[n + 1];
        Arrays.fill(memo, -1);
        return solveMemo(n, memo);
    }

    private static int solveMemo(int n, int[] memo) {
        if (n == 0) return 0;
```

```
        if (memo[n] != -1) return memo[n];

        int minSquares = Integer.MAX_VALUE;
        for (int x = 1; x * x <= n; x++) {
            int count = 1 + solveMemo(n - x * x, memo);
            minSquares = Math.min(minSquares, count);
        }

        return memo[n] = minSquares;
    }

    // Approach 2: Bottom-Up Tabulation O(N sqrt(N)) Time, O(N) Space
    public static int numSquaresTabulation(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0; // Base Case

        for (int i = 1; i <= n; i++) {
            for (int x = 1; x * x <= i; x++) {
                dp[i] = Math.min(dp[i], 1 + dp[i - x * x]);
            }
        }

        return dp[n];
    }
}
```

6 Case Study IV: 2D Grid Unique Paths

Multi-dimensional dynamic programming extends state representation from single indices to multi-variable tuples. The Grid Unique Paths problem models two-dimensional spatial movement.

6.1 Problem Formalization

Given an $N \times M$ grid, an agent starts at the top-left cell $(0, 0)$ and aims to reach the bottom-right cell $(N-1, M-1)$. At any cell (i, j), the agent can move only either Right to $(i, j+1)$ or Down to $(i+1, j)$. Calculate the total number of unique paths from start to destination.

6.2 Recurrence Formulation and Base Cases

Let $P(i, j)$ represent the number of unique paths to reach cell (i, j) from $(0, 0)$.

Theorem 6 (2D Grid Path Recurrence). Any path reaching cell (i, j) must arrive either from cell $(i-1, j)$ via a Down step or from cell $(i, j-1)$ via a Right step:

$$P(i, j) = P(i-1, j) + P(i, j-1) \quad \text{for } i > 0, j > 0$$

Boundary base cases:

$$P(0, j) = 1 \quad \forall j \in [0 \dots M-1], \quad P(i, 0) = 1 \quad \forall i \in [0 \dots N-1]$$

6.3 Combinatorial Analytical Closed-Form Proof

Theorem 7 (Combinatorial Solution for Unique Grid Paths). The exact total number of unique paths on an $N \times M$ grid is given by the binomial coefficient:

$$P(N-1, M-1) = \binom{(N-1)+(M-1)}{N-1} = \frac{(N+M-2)!}{(N-1)!(M-1)!}$$

Proof. To reach $(N-1, M-1)$ from $(0, 0)$, every valid path must execute exactly $(N-1)$ Down moves and $(M-1)$ Right moves, requiring a total of $K = (N-1) + (M-1) = N + M - 2$ total steps. Any path is uniquely specified by choosing which $(N-1)$ of the K steps are Down moves. The number of such choices is given by $\binom{N+M-2}{N-1}$. □

6.4 Compilable Java Implementation for Grid Unique Paths

```
public class GridUniquePaths {

    // Tabulation 2D DP: O(N * M) Time, O(N * M) Space
    public static int uniquePathsTabulation(int m, int n) {
        int[][] dp = new int[m][n];

        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }
        return dp[m - 1][n - 1];
    }

    // Space-Optimized 1D DP: O(N * M) Time, O(N) Space
    public static int uniquePathsSpaceOptimized(int m, int n) {
        int[] dp = new int[n];
        java.util.Arrays.fill(dp, 1);

        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[j] += dp[j - 1];
            }
        }
        return dp[n - 1];
    }
}
```

7 Master Comparative Taxonomy & Algorithmic Synthesis

Table 3 synthesizes all four analyzed case studies, highlighting their state definitions, recurrences, time/space bounds, and optimization strategies.

TABLE 3
Master Algorithmic Taxonomy of Evaluated Dynamic Programming Problems

Problem Domain	Recurrence Relation	Naive Time / Space	DP Time / Space	Optimal Space Reduction
Fibonacci Sequence	$F(n) = F(n-1)+F(n-2)$	$O(2^N)/O(N)$	$O(N)/O(N)$	$O(1)$ space using 2 scalar variables
Climbing Stairs	$W(n) = W(n-1)+W(n-2)$	$O(2^N)/O(N)$	$O(N)/O(N)$	$O(1)$ space using 2 scalar variables
Min Perfect Squares	$S(n) = 1+\min_x S(n-x^2)$	$O(N^N)/O(N)$	$O(N\sqrt{N})/O(N)$	$O(N)$ space required due to non-local look-back
2D Grid Unique Paths	$P(i,j) = P(i-1,j) + P(i,j-1)$	$O(2^{N+M})/O(N+M)$	$O(N \cdot M)/O(N \cdot M)$	$O(\min(N,M))$ space using 1D row array

8 Conclusion

This paper has presented a formal, unified analysis of foundational Dynamic Programming paradigms derived from classroom instruction. By introducing a dedicated mathematical notation and calculus reference guide, analyzing optimal substructure and overlapping subproblem invariants, and evaluating Fibonacci, Climbing Stairs, Minimum Perfect Squares, and Grid Unique Paths problems, we demonstrated the systematic reduction of exponential naive complexities into linear, polynomial, and combinatorial bounds. Production-grade Java implementations were provided for all discussed algorithms.